

مستند DevOps سامانه SaaS Chat AI

بخش ۱۲ راهنمای مستندات پروژه

نسخه سند: ۰.۱

تاریخ تدوین: ۱۵ جولای ۲۰۲۶

تهیه کننده: مبین یوسفی

مسیر خروجی: doc/LaTeX/12-DevOps/12-DevOps.tex

منابع بررسی شده: ArchitectureForMobin.md، Rahnama.md،

ReportForSale.md، README.md و کدهای پروژه

مستند راهبری توسعه، build، deployment، rollback، logging monitoring و آماده سازی production

فهرست مطالب

۱	هدف و دامنه سند	۲
۲	نمای کلی اجزای قابل استقرار	۲
۳	وضعیت فعلی Repository	۲
۱.۳	فایل‌ها و ابزارهای موجود	۲
۲.۳	فایل‌ها و فرآیندهای غایب	۳
۴	محیط‌های عملیاتی	۳
۱.۴	متغیرهای محیطی backend	۳
۲.۴	متغیرهای frontend	۴
۵	اجرای محلی	۴
۱.۵	پیش‌نیازها	۴
۲.۵	راه‌اندازی backend	۴
۳.۵	راه‌اندازی frontend	۵
۴.۵	راه‌اندازی widget	۵
۶	Git و Strategy Branch	۵
۱.۶	قواعد پایه Git	۵
۲.۶	Flow Git پیشنهادی	۵
۳.۶	Request Merge	۶
۷	Release و Versioning	۶
۸	Artifact و Build	۶
۱.۸	Backend	۷
۲.۸	Frontend	۷
۳.۸	Widget	۷
۹	Docker و Compose Docker	۷
۱.۹	وضعیت فعلی	۷
۲.۹	طرح پیشنهادی Docker	۷
۳.۹	طرح پیشنهادی Compose Docker	۸

۸	Kubernetes ۱۰
۹	CI/CD ۱۱
۹	۱.۱۱ وضعیت فعلی
۹	۲.۱۱ Pipeline پیشنهادی
۹	۳.۱۱ نمونه دستورات CI
۱۰	Deployment ۱۲
۱۰	۱.۱۲ Deployment پیشنهادی روی VM
۱۰	۲.۱۲ مسیرهای پیشنهادی production
۱۰	Migration Database ۱۳
۱۱	Rollback ۱۴
۱۱	۱.۱۴ اصول rollback
۱۱	۲.۱۴ سناریوهای rollback
۱۱	Monitoring ۱۵
۱۱	۱.۱۵ Check Health
۱۲	۲.۱۵ شاخص‌های مانیتورینگ
۱۲	Logging ۱۶
۱۲	۱.۱۶ وضعیت فعلی
۱۲	۲.۱۶ پیشنهاد production
۱۳	Tracing ۱۷
۱۳	Deploy Auto ۱۸
۱۳	Backup و نگهداری ۱۹
۱۳	چک‌لیست Production Release ۲۰
۱۴	جمع‌بندی ۲۱

۱ هدف و دامنه سند

این سند فرآیندهای DevOps سامانه SaaS Chat AI را برای توسعه، deployment، release، build، monitoring rollback، و نگهداری عملیاتی توضیح می‌دهد. سند بر اساس وضعیت واقعی repository نوشته شده است: backend با PHP خام، frontend با widget Next.js، به صورت JavaScript قابل انتشار، پایگاه داده MySQL و تنظیمات محیطی مبتنی بر ..env.

در وضعیت فعلی، فایل رسمی برای Docker، Docker Compose، Kubernetes یا CI/CD در repository وجود ندارد. بنابراین این سند دو بخش را همزمان پوشش می‌دهد: وضعیت موجود و الگوی پیشنهادی برای production. هرچا قابلیت هنوز پیاده‌سازی نشده، صریحا با عنوان «پیشنهادی» یا «نیازمند تکمیل» مشخص شده است.

۲ نمای کلی اجزای قابل استقرار

جزء	مسیر/فناوری	مسئولیت عملیاتی
API Backend	MySQL PDO، PHP، backend/	سرو، ها، API احراز هویت، ویجت، پنل، upload AI، دسترسی دیتابیس.
Panel Frontend	React Next.js، frontend/	پنل super، admin، build agent و اجرای Node/Next.
Widget	widget/dist/widget.js	فایل JavaScript قابل embed روی سایت مشتری؛ باید از دامنه یا CDN پایدار سرو شود.
Database	MySQL با های migration SQL	نگهداری، ها، tenant کاربران، سایت‌ها، گفتگوها، دانش، rate AI، audit. و limit
Uploads	backend/uploads یا UPLOAD_STORAGE_ROOT	نگهداری پیوست‌ها و تصاویر؛ نیازمند backup و hardening وب سرور.
Shop Test	test-shop/ و widget/test.html	محیط تست/دمو برای نصب ویجت و سناریوهای نمایشی.

۳ وضعیت فعلی Repository

۱.۳ فایل‌ها و ابزارهای موجود

- frontend/package.json: شامل اسکریپت‌های dev، build، start و lint.
- frontend/next.config.ts: هدرهای امنیتی frontend را تعریف می‌کند.
- backend/.env.example: نمونه متغیرهای محیطی، backend دیتابیس، upload JWT و CORS.
- backend/.htaccess: جلوگیری از directory listing، محافظت از فایل‌های env/sql/log/backup.

- و خاموش کردن errors display در mod_php.
- migration :backend/database/migrations/2026_07_07_ai_knowledge_tables.sql
مربوط به جدول‌های دانش AI.
- check health :backend/api/test-db.php ساده برای اتصال دیتابیس و شمارش جدول plans.
- widget/src/widget.js و widget/dist/widget.js: سورس و خروجی قابل انتشار ویجت.

۲.۳ فایل‌ها و فرآیندهای غایب

- Dockerfile و docker-compose.yml در repository وجود ندارد.
- pipeline رسمی CI/CD مانند GitHub Actions، CI GitLab یا Jenkinsfile وجود ندارد.
- runner migration خودکار دیده نشد؛ ها migration باید فعلاً دستی یا با اسکریپت جدا اجرا شوند.
- check health جامع برای همه سرویس‌ها وجود ندارد؛ فقط test-db.php برای دیتابیس موجود است.
- logging centralized و tracing monitoring، repo پیاده‌سازی نشده‌اند.

۴ محیط‌های عملیاتی

محیط	هدف	قواعد پیشنهادی
Local	توسعه روی ماشین برنامه‌نویس	APP_DEBUG=true, APP_ENV=local localhost. دیتابیس محلی، CORS برای
Staging	تست قبل از انتشار	داده تست، دامنه جدا، APP_DEBUG=false، اجرای migration و test. smoke
Production	سرویس واقعی مشتریان	HTTPS اجباری، رازهای قوی، backup، monitoring محدودیت CORS و rollback آماده.

۱.۴ متغیرهای محیطی backend

متغیرهای اصلی از backend/.env خوانده می‌شوند. فایل env. نباید commit شود و فقط env.example. باید در repository باقی بماند.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://app.example.com
FRONTEND_URL=https://app.example.com
API_URL=https://api.example.com/backend/api
WIDGET_SCRIPT_URL=https://cdn.example.com/widget.js

DB_HOST=127.0.0.1
DB_NAME=ai_chat_saas
DB_USER=ai_chat_saas_user
```

```
DB_PASS=strong-password

JWT_SECRET=long-random-secret-at-least-32-characters
PANEL_ALLOWED_ORIGINS=https://app.example.com
WIDGET_ALLOWED_ORIGINS=https://customer-one.com
WIDGET_ALLOW_EMPTY_ORIGIN=false

UPLOAD_STORAGE_ROOT=/var/www/ai-chat-saas-storage
UPLOAD_PUBLIC_URL=https://api.example.com
```

۲.۴ متغیرهای frontend

frontend از NEXT_PUBLIC_API_BASE_URL استفاده می‌کند. اگر این مقدار تنظیم نشود، مقدار local زیر استفاده می‌شود:

```
http://localhost/ai-chat-saas/backend/api
```

برای: production

```
NEXT_PUBLIC_API_BASE_URL=https://api.example.com/backend/api
```

۵ اجرای محلی

۱.۵ پیش‌نیازها

- PHP با افزونه‌های PDO و MySQL.
- MariaDB یا MySQL.
- Node.js و npm برای frontend.
- وب‌سرور محلی مانند Apache/XAMPP/WAMP یا PHP server built-in برای backend.

۲.۵ راه‌اندازی backend

۱. فایل backend/.env.example به backend/.env کپی و مقادیر دیتابیس تنظیم شود.
۲. دیتابیس ai_chat_saas ساخته شود.
۳. پایه و های migration موجود اجرا شوند. در repo فعلی migration دانش AI در backend/database/migrations قرار دارد.
۴. API از طریق وب‌سرور محلی در مسیری مانند http://localhost/ai-chat-saas/backend/api سرو شود.
۵. اتصال دیتابیس با endpoint زیر تست شود:

```
GET /backend/api/test-db.php
```

۳.۵ راه‌اندازی frontend

```
frontend cd
install npm
dev run npm
```

production: build برای

```
frontend cd
ci npm
build run npm
start run npm
```

۴.۵ راه‌اندازی widget

ویجت فعلی ابزار build جداگانه ندارد و فایل قابل انتشار در widget/dist/widget.js موجود است. در محیط توسعه می‌توان از widget/test.html یا embed زیر استفاده کرد:

```
<script
src="https://cdn.example.com/widget.js"
-key="SITE_PUBLIC_KEY"-sitedata
>-base="https://api.example.com/backend/api"-apidata
></script
```

۶ Git و Strategy Branch

۱.۶ قواعد پایه Git

- شاخه اصلی پایدار باید main یا master باشد.
- تغییرات مستقیم روی شاخه production ممنوع باشد.
- هر قابلیت یا اصلاح با branch جدا و request merge انجام شود.
- فایل‌های .env، dump، دیتابیس، ها، upload ها، log و های build موقت commit نشوند.
- پیام commit باید کوتاه، دقیق و قابل ردیابی باشد.

۲.۶ Flow Git پیشنهادی

کاربرد	Branch
merge از طریق release، فقط از طریق request و پس از تایید.	main
نسخه production یا آماده.	

کاربرد	Branch
تجميع تغييرات آماده. staging. اگر تیم کوچک است، می‌تواند حذف شود و فقط branch feature به merge main شود.	develop
توسعه قابلیت جدید مانند feature/ai-crawl-monitoring.	feature/*
رفع های bug عادی.	fix/*
اصلاح فوری production با مسیر کوتاه‌تر و release سریع.	hotfix/*
آماده‌سازی نسخه، freeze تغییرات، اجرای تست و اصلاحات پایانی.	release/*

۳.۶ Request Merge

هر request merge باید حداقل شامل موارد زیر باشد:

- شرح تغییر و دلیل آن.
- فهرست endpointها یا صفحات تحت تاثیر.
- وضعیت migration دیتابیس.
- نتیجه build/lint/test.
- ریسک rollback.
- تایید اینکه secret یا داده حساس commit نشده است.

۷ Release و Versioning

برای محصول SaaS، نسخه‌بندی پیشنهادی Semantic Versioning است:

```
MAJOR.MINOR.PATCH
1.4.2
```

- MAJOR: تغییر ناسازگار در API، دیتابیس یا قرارداد widget.
- MINOR: قابلیت جدید سازگار با نسخه قبل.
- PATCH: hardening fix، bug یا تغییر بدون اثر روی قرارداد عمومی.

هر release باید tag داشته باشد:

```
git tag -a v1.0.0 -m "Release v1.0.0"
git push origin v1.0.0
```

۸ Artifact و Build

Backend ۱.۸

backend فعلی build ندارد، اما artifact آن باید شامل فایل‌های PHP، backend/api، backend/includes، backend/config، backend/public، migrationها و فایل htaccess باشد. فایل‌های زیر نباید وارد artifact شوند:

- backend/.env
- backend/uploads
- backend/storage/logs
- های dump، SQL log و ها، backup

Frontend ۲.۸

frontend باید با build npm شود:

```
cd frontend
npm ci
npm run build
```

Artifact frontend شامل .next، public، package.json و package-lock.json است. برای اجرای production:

```
npm run start
```

Widget ۳.۸

در وضعیت فعلی، widget/dist/widget.js artifact widget/ نهایی ویجت است. اگر minify/bundle رسمی اضافه شود، باید خروجی dist در CI تولید و checksum آن ثبت شود. URL نهایی ویجت باید با WIDGET_SCRIPT_URL هماهنگ باشد.

۹ Docker و Compose Docker

۱.۹ وضعیت فعلی

در repository فایل Dockerfile یا docker-compose.yml وجود ندارد. بنابراین deployment فعلی احتمالاً به وب‌سرور سنتی PHP و سرویس Node برای Next.js وابسته است.

۲.۹ طرح پیشنهادی Docker

برای production یا staging قابل تکرار، سه container اصلی پیشنهاد می‌شود:

- backend-php: PHP-FPM یا Apache/PHP برای APIها.

- Next.js: برای اجرای Node.js
- mysql: دیتابیس فقط برای local/staging؛ در production بهتر است سرویس managed یا VM جدا باشد.

نمونه ساختار پیشنهادی:

```
docker/
  backend/Dockerfile
  frontend/Dockerfile
  nginx/default.conf
docker-compose.yml
```

۳.۹ طرح پیشنهادی Compose Docker

```
services:
  backend:
    build: ./docker/backend
    env_file: ./backend/.env
    volumes:
      - ./backend:/var/www/backend
      - uploads:/var/www/storage/uploads

  frontend:
    build: ./docker/frontend
    environment:
      NEXT_PUBLIC_API_BASE_URL: https://api.example.com/backend/api

  mysql:
    image: mysql:8
    environment:
      MYSQL_DATABASE: ai_chat_saas
      MYSQL_USER: app
      MYSQL_PASSWORD: secret
      MYSQL_ROOT_PASSWORD: root-secret

volumes:
  uploads:
```

این نمونه در repo فعلی وجود ندارد و باید قبل از استفاده واقعی با نیازهای مسیر فایل، وب سرور و SSL تطبیق داده شود.

۱۰ Kubernetes

Kubernetes برای MVP ضروری نیست، اما برای SaaS با چند مشتری و نیاز به مقیاس پذیری می تواند در فاز بعدی استفاده شود. اجزای پیشنهادی:

- Deployment برای frontend و backend.

- Service جدا برای frontend و backend.
- Ingress با TLS و routing دامنه‌ها.
- Secret برای JWT_SECRET، DB_PASS و کلیدهای API.
- ConfigMap برای URLها و تنظیمات غیرحساس.
- PersistentVolume یا storage object برای .upload
- CronJob برای backup، cleanup و taskهای دوره‌ای.

تا زمانی که load و تیم عملیاتی کوچک است، deployment روی VM با Docker Compose ساده‌تر و کم‌ریسک‌تر از Kubernetes است.

۱۱ CI/CD

۱.۱۱ وضعیت فعلی

pipeline رسمی در repository وجود ندارد. بنابراین اجرای build، lint، migration و deploy فعلاً باید دستی یا با ابزار بیرونی انجام شود.

۲.۱۱ Pipeline پیشنهادی

۱. Checkout کد.
۲. نصب dependencyهای frontend با `npm ci`.
۳. اجرای frontend lint با `npm run lint`.
۴. اجرای frontend build با `npm run build`.
۵. بررسی syntax فایل‌های PHP.
۶. اجرای تست‌های backend/frontend در صورت اضافه شدن.
۷. ساخت artifact، backend، frontend و widget.
۸. deploy به staging.
۹. اجرای migration روی staging.
۱۰. test smoke شامل login، health check، دیتابیس، config widget و یک endpoint محافظت‌شده.
۱۱. approval دستی برای production.
۱۲. deploy production با نسخه tag شده.
۱۳. check health post-deploy و مانیتورینگ خطا.

۳.۱۱ نمونه دستورات CI

```
cd frontend
npm ci
npm run lint
npm run build

php -l backend/api/auth/login.php
php -l backend/includes/jwt.php
```

برای بررسی همه فایل‌های PHP، در CI می‌توان از اسکریپت shell/PowerShell استفاده کرد. این اسکریپت باید روی همه فایل‌های backend/**/*.php اجرا شود و در صورت خطای pipeline syntax را متوقف کند.

۱۲ Deployment

۱.۱۲ Deployment پیشنهادی روی VM

۱. دریافت artifact نسخه مشخص از CI یا checkout tag.
۲. نصب های frontend dependency با `npm ci --omit=dev` یا استفاده از build artifact شده.
۳. کپی backend به مسیر وب سرور.
۴. قرار دادن `backend/.env` امن خارج از repository یا تزریق از `manager.secret`.
۵. اجرای های migration دیتابیس با backup قبل از migration.
۶. تنظیم `UPLOAD_STORAGE_ROOT` و `permission` مناسب.
۷. restart سرویس Node/Next و PHP/Apache/Nginx در صورت نیاز.
۸. اجرای `test.smoke`.

۲.۱۲ مسیرهای پیشنهادی production

کاربرد	دامنه/مسیر
Next.js. پنل frontend	<code>https://app.example.com</code>
API. PHP	<code>https://api.example.com/backend/api</code>
فایل static ویجت.	<code>https://cdn.example.com/widget.js</code>
کد backend روی سرور.	<code>/var/www/ai-chat-saas/backend</code>
کد یا artifact frontend.	<code>/var/www/ai-chat-saas/frontend</code>
storage upload خارج از مسیر اجرای کد.	<code>/var/www/ai-chat-saas-storage</code>

۱۳ Migration Database

در repo فعلی فقط migration دانش AI دیده شد و runner migration مرکزی وجود ندارد. برای pro-duction باید سیاست زیر اجرا شود:

- هر تغییر schema در فایل SQL نسخه دار داخل `backend/database/migrations` ثبت شود.
- قبل از اجرای migration در production، backup دیتابیس گرفته شود.
- migration ابتدا در staging اجرا شود.
- های destructive migration باید rollback plan یا اسکریپت جبران داشته باشند.
- جدول `schema_migrations` برای ثبت های migration اجرا شده اضافه شود.

نمونه اجرای دستی:

```
backend/database/migrations/2026_07_07_ai_knowledge_tables.sql < ai_chat_saas -p app -u mysql
```

۱۴ Rollback

۱.۱۴ اصول rollback

Rollback باید قبل از deploy طراحی شود، نه بعد از شکست. برای هر release باید موارد زیر مشخص باشد:

- tag نسخه قبلی سالم.
- artifact نسخه قبلی، frontend backend و widget.
- وضعیت migration و امکان rollback دیتابیس.
- سازگاری نسخه قدیمی frontend با API و دیتابیس جدید.
- backup قبل از migration.

۲.۱۴ سناریوهای rollback

ریسک	اقدام	سناریو
پایین، اگر API تغییر نکرده باشد.	بازگرداندن artifact قبلی Next.js و restart سرویس.	خرابی frontend
متوسط، وابسته به migration.	بازگرداندن کد PHP قبلی و پاک‌سازی cache وب‌سرور در صورت وجود.	خرابی backend
متوسط، چون روی سایت مشتری cache می‌شود.	بازگرداندن widget قبلی در CDN و کاهش TTL cache.	خرابی widget
بالا، باید از قبل تست شود.	migration یا backup restore اجرا و جبرانی.	خرابی migration
متوسط، نیازمند audit.	بازگرداندن env قبلی از secret manager و restart سرویس‌ها.	مشکل env/secret

۱۵ Monitoring

۱.۱۵ Check Health

در حال حاضر backend/api/test-db.php check health ساده دیتابیس است. برای production بهتر است endpoint اختصاصی /health اضافه شود که بدون افشای اطلاعات حساس موارد زیر را بررسی کند:

- دسترسی runtime. PHP
- اتصال دیتابیس با query سبک.

- writable بودن upload storage در صورت نیاز.
- وضعیت نسخه build.
- عدم نمایش trace stack یا جزئیات secret.

نمونه پاسخ پیشنهادی:

```
{
  "success": true,
  "status": "ok",
  "version": "1.0.0",
  "checks": {
    "database": "ok",
    "storage": "ok"
  }
}
```

۲.۱۵ شاخص‌های مانیتورینگ

- نرخ خطاهای HTTP ۴xx و ۵xx
- latency های endpoint، login، conversation و send message list، AI reply.
- تعداد گفتگوهای ایجادشده در دقیقه.
- تعداد رخدادهای limit. rate
- خطاهای upload و حجم storage.
- مصرف CPU/RAM دیپلوی frontend و backend.
- اندازه دیتابیس و سرعت رشد جدول‌های logs، audit و conversations messages.
- تعداد خطاهای crawler و AI search.

۱۶ Logging

۱.۱۶ وضعیت فعلی

backend از error_log برای خطاهای امنیتی و runtime استفاده می‌کند. audit مدیریتی نیز در جدول admin_audit_logs ذخیره می‌شود. در repository، مسیرهای backend/storage/logs قرار دارند. gitignore.

۲.۱۶ پیشنهاد production

- لاگ، PHP و وب‌سرور، Node/Next و دیتابیس به aggregator log مرکزی ارسال شود.
- ها log ساختاریافته و شامل id tenant id، user id، request و endpoint باشند.
- secret، token password و Authorization header هرگز در log ذخیره نشوند.
- retention لاگ‌ها بر اساس سیاست تجاری و حریم خصوصی تعریف شود.
- alert برای رشد خطای ۵xx، شکست login غیرعادی، limit rate شدید و خطاهای upload فعال شود.

۱۷ Tracing

Tracing توزیع شده در repo فعلی پیاده سازی نشده است. با توجه به معماری فعلی، ابتدا logging ساختاریافته و id request کافی است. در مرحله بعد می توان OpenTelemetry یا ابزار مشابه را برای ردیابی مسیر درخواست بین backend frontend، و دیتابیس اضافه کرد. حداقل طراحی پیشنهادی:

- تولید X-Request-Id در proxy reverse اگر client نفرستاده باشد.
- عبور دادن id request به PHP و Next.js و logs.
- ثبت id request در log error و context.audit.
- نمایش id request در پاسخ خطای production برای پشتیبانی، بدون افشای جزئیات فنی.

۱۸ Deploy Auto

deploy Auto فعلا در repo پیاده سازی نشده است. پیشنهاد:

- merge به develop: deploy خودکار به staging.
- tag با الگوی v*: ساخت artifact.release
- production deploy فقط بعد از approval دستی.
- test smoke خودکار پس از deploy.
- rollback دستی اما سریع با artifact نسخه قبلی.

۱۹ Backup و نگهداری

- backup روزانه دیتابیس و نگهداری چند نسخه زمانی.
- backup هماهنگ storage upload با دیتابیس.
- تست restore حداقل ماهانه.
- پاک سازی دوره ای، logهای فایل های موقت و rate های limit منقضی.
- مانیتور فضای دیسک upload و backup.
- نگهداری نسخه schema و های migration اجرا شده.

۲۰ چک لیست Production Release

۱. branch با release tag مشخص شده است.
۲. frontend با npm ci و npm run build موفق build شده است.
۳. فایل های PHP syntax check شده اند.
۴. migration ها در staging اجرا و تایید شده اند.
۵. backup دیتابیس و storage upload گرفته شده است.
۶. APP_DEBUG=false و APP_ENV=production تنظیم شده است.
۷. JWT_SECRET و DB_PASS از secret امن تامین شده اند.
۸. WIDGET_ALLOW_EMPTY_ORIGIN=false و PANEL_ALLOWED_ORIGINS کنترل شده اند.

۹. check health و test smoke پس از deploy موفق هستند.

۱۰. برنامه rollback و artifact نسخه قبلی آماده است.

۲۱ جمع‌بندی

DevOps فعلی پروژه برای توسعه و دمو قابل استفاده است، اما برای production نیازمند تکمیل است. اجزای اصلی محصول مشخص و قابل deploy هستند: backend PHP، frontend Next.js، widget static، MySQL و storage. upload با این حال، نبود Docker، نبود pipeline رسمی، نبود CI/CD، نبود runner، migration، monitoring/tracing مرکزی و نبود check health جامع باید قبل از عرضه گسترده برطرف شود. مسیر پیشنهادی، ابتدا استانداردسازی env و release دستی قابل تکرار، سپس Compose Docker برای staging/production کوچک، و در نهایت CI/CD، monitoring و در صورت نیاز Kubernetes است.